



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE

Rust 课程期末报告

〈基于 Rust 的多线程日志分析 CLI 工具〉

姓名：吴逸凡

学号：2411522

2026 年 6 月 14 日

一、项目简介

本项目是一个基于 Rust 语言实现的多线程日志分析 CLI 工具，主要面向服务器运行日志、应用程序日志等文本型日志数据。程序能够读取指定日志文件，解析每一行日志中的服务名称、日志等级、延迟信息和消息内容，并进一步统计不同日志等级数量、不同服务的日志数量、错误日志分布和慢请求情况。

在实际软件系统中，日志是排查问题和观察系统运行状态的重要依据。当日志数量较多时，人工逐行查看效率较低，也容易遗漏异常信息。因此，本项目希望通过自动化日志解析、统计分析和报告导出，帮助用户快速了解日志整体情况，并定位错误日志和慢请求较集中的服务。

项目支持命令行运行，用户可以通过 `cargo run -- demo` 运行内置示例，也可以通过 `cargo run -- analyze examples/sample.log` 分析指定日志文件。在迭代版本中，项目新增了报告导出功能，支持将分析结果导出为 Markdown、JSON 或 CSV 等格式，方便后续保存和查看。

GitHub 链接: https://github.com/Forgetme-Not125/rust_log_lab

二、小组成员分工

单人项目，不涉及小组成员分工。

三、项目结构

rust_log_lab/	
—— Cargo.toml	Cargo.toml: Rust 项目配置和依赖说明
—— Cargo.lock	README.md: 项目说明文档
—— README.md	examples/sample.log: 示例日志文件
—— .gitignore	docs/: 实验报告和迭代说明
—— examples/	src/: Rust 源码目录
—— sample.log	main.rs: 程序入口
—— docs/	lib.rs: 模块统一声明
—— 实验报告.docx	cli.rs: 命令行参数解析
—— 迭代版本说明.md	model.rs: 核心数据结构
—— src/	parser.rs: 日志解析
—— main.rs	analyzer.rs: 多线程分析与统计
—— lib.rs	report.rs: 报告渲染
—— cli.rs	export.rs: 报告导出
—— model.rs	error.rs: 统一错误处理
—— parser.rs	
—— analyzer.rs	
—— report.rs	
—— export.rs	
—— error.rs	

四、设计与实现

本项目整体采用“输入解析—日志解析—并发分析—报告生成—结果导出”的设计流程。用户首先通过命令行输入操作命令，例如运行内置示例、分析指定日志文件、设置线程数量、筛选日志等级或指定报告导出路径。程序在解析命令行参数后，根据用户选择读取内置 demo 数据或本地日志文件。

日志读取完成后，程序会逐行解析日志内容。每一行日志会被解析为结构化的日志记录对象，其中包含服务名称、日志等级、延迟时间和消息内容等字段。对于格式不合法的日志，程序不会直接崩溃，而是通过错误处理机制记录解析失败情况。

在分析阶段，项目使用多线程方式处理日志数据。程序会根据设置的线程数量将

日志数据拆分为多个片段,每个工作线程独立完成局部日志统计。各线程完成后,主线程会收集并合并局部统计结果,最终得到完整的全局分析结果。

在报告生成阶段,程序会根据统计结果生成日志总数、解析成功数、错误日志数量、慢请求数量、日志等级分布、服务排行和健康评估信息。最后,程序可以将报告输出到终端,也可以通过 `--out` 参数导出为 Markdown、JSON 或 CSV 文件。

这种设计将命令行解析、数据建模、日志解析、统计分析、报告渲染和文件导出分离到不同模块中,使项目结构更加清晰,也便于后续扩展。

五、各模块详细说明

5.1 main.rs

内容: 程序入口、调用 CLI 解析、根据用户命令执行 `demo` 或 `analyze`、统一处理错误

`main.rs` 是项目入口模块,主要负责启动程序并组织整体执行流程。程序首先调用命令行解析模块获取用户输入的命令和参数,然后根据不同命令选择执行内置 `demo` 或分析指定日志文件。同时,主函数通过 `Result` 处理运行过程中可能出现的错误,例如参数错误、文件读取错误或报告导出错误。

5.2 cli.rs

内容: 解析 `analyze/demo`、解析 `--out`、`--format`、`--threads`、`--level` 等参数、提供 `usage` 帮助信息

`cli.rs` 模块负责命令行参数解析。项目支持 `demo` 和 `analyze` 两类主要命令,前者用于运行内置演示数据,后者用于分析用户指定的日志文件。同时,该模块还支持解析 `--threads`、`--level`、`--service`、`--keyword`、`--format` 和 `--out` 等可选参数。通过单独设置 CLI 模块,项目将用户交互逻辑与日志分析逻辑分离。如果后续需要增加新的命令或参数,只需要在该模块中扩展。

5.3 model.rs

内容: LogRecord、LogLevel、FilterOptions、OutputFormat

model.rs 模块定义项目中的核心数据结构。日志记录使用结构体表示，日志等级使用枚举类型表示。通过 LogRecord 结构体，程序可以将原始字符串形式的日志转换为包含服务名、等级、延迟和消息内容的结构化数据。

日志等级使用 enum 表示，例如 DEBUG、INFO、WARN、ERROR 和 FATAL。相比直接使用字符串，枚举类型更加安全，能够避免拼写错误造成的逻辑问题。此外，项目还定义了输出格式枚举，用于表示 text、markdown、json、csv 等不同报告格式。同时，该模块体现了 Rust 中 struct、enum、方法实现和类型安全设计等特性。

5.4 parser.rs

内容: 解析 service=auth level=ERROR latency=120 msg="login failed"、处理字段缺失、返回 Result

parser.rs 模块负责将原始日志文本解析为结构化日志记录。程序会按行读取日志内容，并从每一行中提取 service、level、latency 和 msg 等字段。解析成功后，程序会构造对应的 LogRecord 对象，供后续分析模块使用。在解析过程中，日志格式可能不完全合法，例如缺少必要字段、日志等级无法识别或延迟字段不是数字。针对这些情况，解析函数使用 Result 返回解析结果或错误信息，而不是直接让程序崩溃。这体现了 Rust 错误处理机制的优势。本模块主要体现了字符串处理、模式匹配、Option、Result 和错误传播等 Rust 特性。

5.5 analyzer.rs

内容: 多线程处理、chunk 划分、thread、channel、结果合并、HashMap 统计

analyzer.rs 是项目的核心分析模块，负责对日志记录进行统计分析。该模块会统计日志总行数、成功解析数量、跳过数量、解析错误数量、不同日志等级数量、不同服务出现次数、错误日志记录和慢请求记录等信息。

为了体现 Rust 的并发编程能力，项目在该模块中使用多线程方式处理日志数据。程序会根据用户指定的线程数量，将日志数据划分为多个片段，并创建多个

工作线程分别处理。每个线程只处理自己负责的数据片段，并生成局部统计结果。主线程通过 `channel` 接收各个线程返回的结果，再将它们合并为最终的全局统计结果。该模块还使用 `HashMap` 统计服务出现次数和日志等级分布，体现了 Rust 标准集合类型的应用。

5.6 report.rs

内容：终端文本报告、Markdown 报告、JSON / CSV 报告、健康评估、Top N 服务、错误日志和慢请求展示

`report.rs` 模块负责将分析结果渲染为用户可读的报告。根据用户选择的输出格式，该模块可以生成普通文本报告、Markdown 报告、JSON 报告或 CSV 报告。文本报告主要用于终端直接查看，内容包括日志总数、解析成功数、日志等级分布、延迟统计、健康评估、服务排行、错误日志样例和慢请求样例。Markdown 报告则适合保存和后续阅读，结构更加清晰，可以直接作为分析文档查看。

该模块还包含健康评估逻辑，会根据错误日志比例和慢请求比例生成健康状态，例如 OK、WARNING 或 CRITICAL，并给出相应建议。该模块体现了 `trait` 抽象、格式化输出和模块化扩展思想。

5.7 export.rs

本模块是新增模块，内容：负责输出到终端或文件、`--out report.md`、`std::fs::write`、`Result` 错误处理

`export.rs` 是项目迭代版本中新增的报告导出模块。在基础版本中，程序主要将分析结果输出到终端；在迭代版本中，用户可以通过 `--out` 参数指定输出路径，将分析结果保存为本地文件。例如，用户运行 `cargo run -- analyze examples/sample.log --out report.md` 后，程序会在项目目录下生成 Markdown 格式的分析报告。该功能提高了项目的实用性，使日志分析结果可以被保存、归档和后续查看。该模块主要使用 Rust 标准库中的文件写入功能，并通过 `Result` 处理文件创建失败、路径错误或写入失败等情况。

5.8 error.rs

内容：自定义错误类型、文件错误、解析错误、参数错误、导出错误

`error.rs` 模块负责定义项目中的统一错误类型。项目运行过程中可能出现多种错误，例如命令行参数不合法、日志文件读取失败、日志格式错误、报告导出失败等。如果每个模块都单独处理错误，代码会比较混乱。因此，本项目通过统一错误类型对不同错误进行封装，并使用 `Result` 在模块之间传递错误信息。这样既可以避免大量使用 `unwrap` 和 `expect`，也能让程序在出现问题时给出更清晰的错误提示。

六、运行截图

图 1：项目结构截图

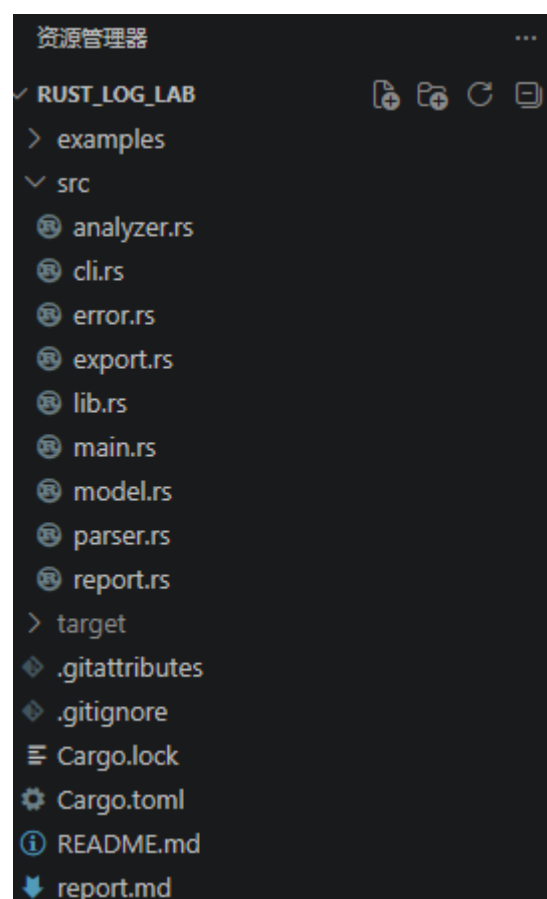
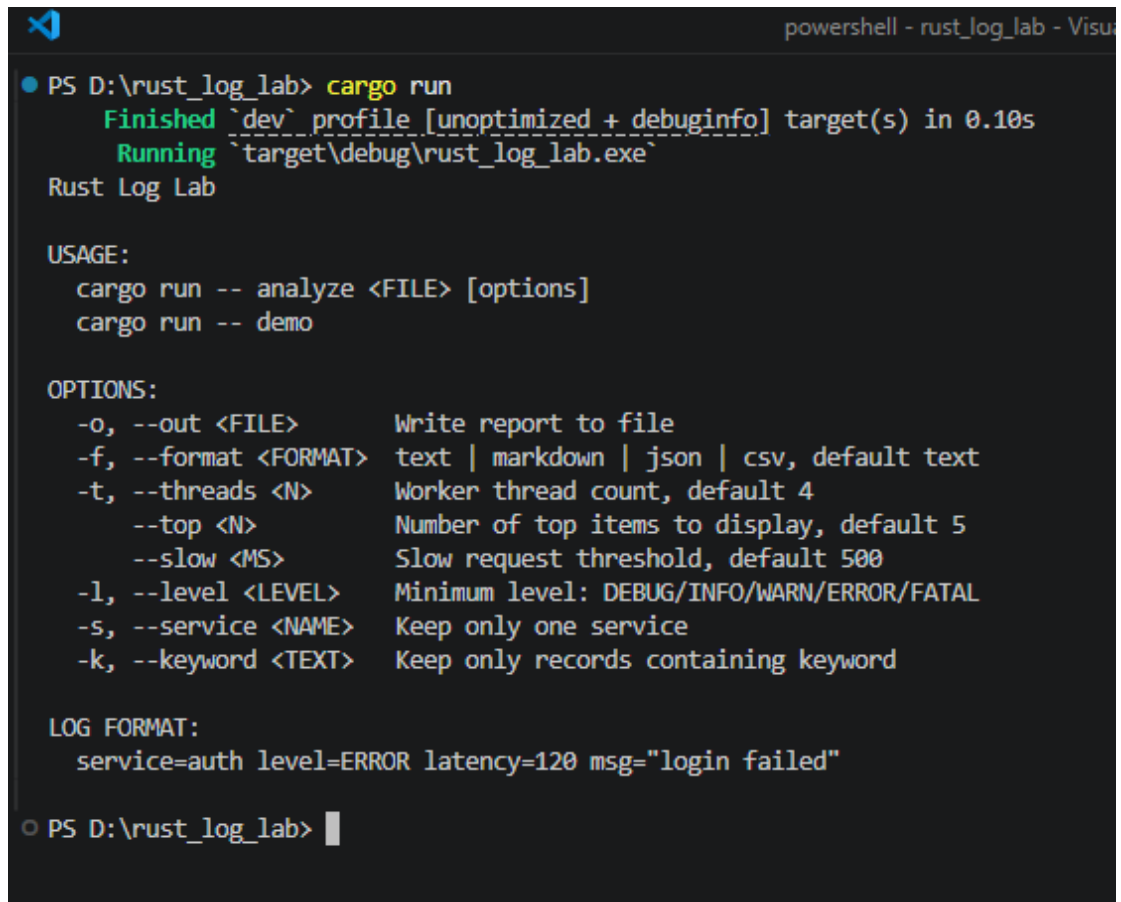


图 1 展示了项目目录结构。项目采用 Cargo 标准工程结构，源代码位于 `src` 目录，示例日志文件位于 `examples` 目录。

图 2：直接运行帮助信息



```
powershell - rust_log_lab - Visual Studio Code
PS D:\rust_log_lab> cargo run
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.10s
Running `target\debug\rust_log_lab.exe`
Rust Log Lab

USAGE:
  cargo run -- analyze <FILE> [options]
  cargo run -- demo

OPTIONS:
  -o, --out <FILE>      Write report to file
  -f, --format <FORMAT> text | markdown | json | csv, default text
  -t, --threads <N>     Worker thread count, default 4
                        --top <N>       Number of top items to display, default 5
                        --slow <MS>     Slow request threshold, default 500
  -l, --level <LEVEL>   Minimum level: DEBUG/INFO/WARN/ERROR/FATAL
  -s, --service <NAME>  Keep only one service
  -k, --keyword <TEXT>  Keep only records containing keyword

LOG FORMAT:
  service=auth level=ERROR latency=120 msg="login failed"

PS D:\rust_log_lab>
```

图 2 展示了程序的命令行帮助信息。用户直接运行程序时，系统会输出支持的命令、参数和日志格式说明。

图 3: 运行内置 demo

```
● PS D:\rust_log_lab> cargo run -- demo
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
    Running `target\debug\rust_log_lab.exe demo`
Rust Log Lab - Analysis Report
=====
total lines      : 7
parsed records   : 7
skipped lines    : 0
parse errors     : 0

Level distribution
  DEBUG 1
  INFO 2
  WARN 2
  ERROR 2

Latency
  samples : 7
  min/max : Some(11)/Some(830) ms
  average : 320.86 ms
  slow    : 3

Health insight
  health : CRITICAL
  score  : 40
  errors : 28.6%
  slow   : 42.9%
  main svc: auth
  - 错误日志占比为 28.6%，建议查看错误集中出现的服务。
  - 慢请求占比达到 42.9%，建议检查数据库、缓存和下游接口耗时。

Top 5 services
  auth      2
  order     2
  payment   2
  search    1

Error-like records
  line 2  auth      ERROR 145ms login failed
  line 4  payment   ERROR 830ms payment timeout

Slow records
  line 3  order     WARN 610ms slow query
  line 4  payment   ERROR 830ms payment timeout
  line 7  payment   WARN 540ms retry payment gateway
○ PS D:\rust_log_lab> 
```

图 3 展示了项目内置 demo 的运行结果。程序成功完成日志解析、等级统计、延迟分析、健康评估和异常日志输出。

图 4：分析示例日志文件

```
● PS D:\rust_log_lab> cargo run -- analyze examples/sample.log
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
    Running `target\debug\rust_log_lab.exe analyze examples/sample.log`
Rust Log Lab - Analysis Report
=====
total lines      : 10
parsed records   : 10
skipped lines    : 0
parse errors     : 0

Level distribution
  DEBUG 1
  INFO 3
  WARN 2
  ERROR 4

Latency
  samples : 10
  min/max : Some(14)/Some(1002) ms
  average : 336.90 ms
  slow    : 3

Health insight
  health : CRITICAL
  score  : 37
  errors : 40.0%
  slow   : 30.0%
  main svc: auth
  - 错误日志占比达到 40.0%，建议优先排查 ERROR/FATAL 记录。
  - 慢请求占比为 30.0%，建议进一步按 service 维度定位。

Top 5 services
  auth      3
  order     3
  payment   2
  search    2

Error-like records
  line 2  auth      ERROR 145ms login failed
  line 4  payment   ERROR 830ms payment timeout
  line 5  auth      ERROR 120ms password wrong
  line 10 order     ERROR 1002ms order rollback

Slow records
  line 3  order     WARN 610ms slow query
  line 4  payment   ERROR 830ms payment timeout
  line 10 order     ERROR 1002ms order rollback
```

图 4 展示了程序对本地示例日志文件 examples/sample.log 的分析结果，说明项目能够读取外部日志文件并生成分析报告。

图 5: 筛选 ERROR 日志

```
PS D:\rust_log_lab> cargo run -- analyze examples/sample.log --level ERROR
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
Running `target\debug\rust_log_lab.exe analyze examples/sample.log --level ERROR`
Rust Log Lab - Analysis Report
=====
total lines      : 10
parsed records   : 4
skipped lines    : 0
parse errors     : 0

Level distribution
  ERROR 4

Latency
  samples : 4
  min/max : Some(120)/Some(1002) ms
  average : 524.25 ms
  slow    : 2

Health insight
  health : CRITICAL
  score  : 20
  errors : 100.0%
  slow   : 50.0%
  main svc: auth
  - 错误日志占比达到 100.0%, 建议优先排查 ERROR/FATAL 记录。
  - 慢请求占比达到 50.0%, 建议检查数据库、缓存和下游接口耗时。

Top 5 services
  auth      2
  order     1
  payment   1

Error-like records
  line 2  auth  ERROR 145ms login failed
  line 4  payment ERROR 830ms payment timeout
  line 5  auth  ERROR 120ms password wrong
  line 10 order ERROR 1002ms order rollback

Slow records
  line 4  payment ERROR 830ms payment timeout
  line 10 order ERROR 1002ms order rollback
```

图 5 展示了按日志等级筛选的功能。通过 `--level ERROR` 参数，程序只保留 ERROR 及以上等级的日志进行分析。

图 6: 导出 Markdown 报告

```
PS D:\rust_log_lab> cargo run -- analyze examples/sample.log --out report.md
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.02s
Running `target\debug\rust_log_lab.exe analyze examples/sample.log --out report.md`
```

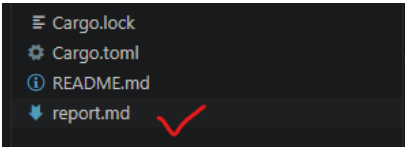


图 6 展示了项目迭代版本新增的 Markdown 报告导出功能。程序通过 `--out report.md` 参数将分析结果写入本地文件，并在项目目录中生成 `report.md`。

图 7：运行测试

```
test result: ok. 18 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

Running unittests src\main.rs (target\debug\deps\rust_log_lab-5cb93cb0ddc051aa.exe)

running 17 tests
test cli::tests::parse_analyze_command ... ok
test cli::tests::output_format_can_be_inferred_from_output_path ... ok
test model::tests::filter_checks_min_level_and_service ... ok
test cli::tests::demo_dataset_covers_multiple_levels ... ok
test parser::tests::parse_quoted_message ... ok
test export::tests::infer_markdown_format_from_extension ... ok
test parser::tests::parser_builds_record ... ok
test parser::tests::parser_ignores_empty_line ... ok
test model::tests::parse_level_is_case_insensitive ... ok
test analyzer::tests::analyze_counts_levels ... ok
test report::tests::insight_is_healthy_when_no_error_or_slow_records ... ok
test analyzer::tests::analysis_result_helper_methods_work ... ok
test report::tests::insight_becomes_risky_when_many_errors_exist ... ok
test report::tests::markdown_report_contains_sections ... ok
test report::tests::text_report_contains_title ... ok
test report::tests::json_report_contains_insight_block ... ok
test export::tests::export_report_writes_file ... ok

test result: ok. 17 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.01s

Doc-tests rust_log_lab

running 0 tests

test result: ok. 0 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finished in 0.00s
```

图 7 展示了项目测试运行结果。测试内容覆盖日志解析、参数解析、统计分析和报告输出等关键功能，测试结果全部通过。

图 8：代码格式化和质量检查

```
● PS D:\rust_log_lab> cargo fmt
● PS D:\rust_log_lab> cargo clippy
    Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.27s
○ PS D:\rust_log_lab>
```

图 8 展示了项目工程规范检查结果。项目通过 `cargo fmt` 进行代码格式化，并通过 `cargo clippy` 进行代码检查，保证代码风格和质量符合要求。

七、遇到的问题与解决方法

在本项目实施过程中，我遇到的主要问题是：如何在多线程环境下安全地完成日志统计，并将多个线程得到的局部统计结果合并成最终的全局分析结果。

项目最初的实现思路比较简单：读取日志文件后，直接用一个循环逐行解析日志，并在同一个统计结果对象中更新日志数量、日志等级分布、服务出现次数、错误日志记录和慢请求记录等信息。这种单线程写法比较容易理解，所有数据都在一个函数中顺序处理，不需要考虑线程之间的数据共享问题。但是，这样的实现不能很好地体现 Rust 在并发编程方面的优势，也不符合本项目“多线程日志分析工具”的目标。

在尝试改成多线程处理时，我一开始想到的是让多个线程共同修改同一个统计结果对象。例如，主线程创建一个全局的 `AnalysisResult`，然后多个工作线程同时向里面写入统计结果。但在实际编写时，我发现这种方式会遇到所有权和借用规则的限制。多个线程如果同时持有同一个可变对象的引用，就可能发生数据竞争，而 Rust 编译器不会允许这种不安全的共享可变访问。

后来我调整了思路：不再让多个线程同时修改同一个全局结果，而是让每个线程只处理自己负责的一部分日志数据，并生成一个局部统计结果。主线程先把日志行按照线程数量划分为若干块，每个线程拿到其中一块数据后独立完成解析和统计。由于每个线程处理的是自己的数据片段，不需要和其他线程同时修改同一个对象，因此可以减少共享状态带来的复杂性。

每个线程完成统计后，会把自己的局部结果发送回主线程。主线程再统一接收这些局部结果，并调用合并逻辑将它们整合为最终的全局分析结果。合并时，需要把日志总数、解析成功数、跳过行数、解析错误数进行累加；对于日志等级分布和服务统计这种 `HashMap` 类型的数据，则需要遍历每个局部结果中的键值对，并把相同键对应的数量累加起来；对于错误日志记录和慢请求记录，则将各线程中的记录追加到最终结果中。

这个问题让我意识到，多线程程序并不是简单地把循环拆成几个线程就可以了，更重要的是要设计好线程之间的数据关系。如果多个线程共享并修改同一个数据结构，就必须处理同步和安全问题；而如果让每个线程独立计算局部结果，最后由主线程统一合并，就可以让程序结构更加清晰，也更符合 Rust 的所有权和并

发安全思想。

最终，本项目采用“数据分块—线程局部分析—主线程合并结果”的方式解决了这个问题。这样既实现了多线程日志分析功能，又避免了多个线程同时修改共享数据带来的风险。

八、其他需要说明的内容

特别鸣谢 ChatGPT。在完成此项目的过程中，我遇到烧脑的问题时会向 GPT 求助，绝大多数情况下，它都能顺利解决问题并给出下一步意见，其实我也怀疑过目前的 ai 这么强，人类程序员是否还有存在的必要。虽然这个问题还有待商榷，但是我不得不承认这是我花费的最值得的 20 美金。

九、总结

通过本次大作业，我对 Rust 语言的理解从单个知识点逐渐扩展到了完整项目开发。以前在课堂学习中，更多是学习变量、所有权、借用、结构体、枚举、错误处理等相对独立的内容，而这次项目需要把这些知识综合起来，完成一个可以运行、可以测试、甚至有点用的命令行工具。

在项目实现过程中，我进一步熟悉了 Rust 的模块化设计、Result 错误处理、struct 和 enum 的使用、多线程处理以及 channel 通信等内容。尤其是在多线程日志分析部分，我意识到并发程序不能简单地把循环拆成多个线程，还需要考虑线程之间的数据关系和统计结果合并方式。

此外，在经过一次迭代后新增报告导出功能时，我也体会到了模块划分的重要性。将报告生成逻辑放在 `report.rs`，将文件导出逻辑放在 `export.rs`，让程序结构更加清晰，也方便后续继续扩展新的输出格式。

总的来说，这次大作业让我完成了从“学习 Rust 语法”到“使用 Rust 完成一个小型工程项目”的过渡。虽然项目还有可以继续完善的地方，但目前它已经具备了较完整的功能流程。通过这次实践，我对 Rust 工程化开发有了更真实的体验。

最后，感谢老师，感谢助教，感谢 ChatGPT。